

Guía docente

Python dentro de un EDA

Cómo explicar sintaxis, objetos, índices, listas, métodos y filtros mientras se escribe código

Enfoque pedagógico: El objetivo no es dictar un curso de Python. La sintaxis aparece cuando la necesitamos para resolver preguntas reales sobre los datos.

Curso: Big Data - Laboratorio
Semana 3 - EDA de Afiliados SNP

1. Modelo mental para leer Python durante un EDA

Antes de memorizar comandos, conviene enseñar a leer una expresión de izquierda a derecha:

```
objeto -> selección -> método -> resultado
```

```
df["sexoact"].value_counts(normalize=True)
```

Idea para decir en clase: No intenten memorizar toda la sintaxis hoy. Intenten aprender a leerla: objeto, selección, operación y resultado.

Concepto	Ejemplo	Lectura en clase
Variable	<code>ruta = "datos.csv"</code>	Un nombre que referencia un valor.
String	<code>"Puno"</code>	Texto. En Python su tipo es <code>str</code> .
Lista	<code>["sexoact", "edadact"]</code>	Colección ordenada; se escribe con corchetes.
Tupla	<code>df.shape</code>	Colección ordenada e inmutable; <code>shape</code> devuelve (filas, columnas).
Diccionario	<code>{1: "Hombre", 2: "Mujer"}</code>	Asocia claves con valores.
Booleano	<code>True / False</code>	Representa verdadero o falso.
DataFrame	<code>df</code>	Tabla de pandas con filas y columnas.
Series	<code>df["sexoact"]</code>	Normalmente una sola columna de pandas.

Regla visual útil

```
objeto.atributo # consulta una propiedad
```

```
objeto.metodo(...) # ejecuta una operación
```

Idea para decir en clase: El punto se puede leer como "algo que pertenece a este objeto". Los paréntesis indican que estamos ejecutando una función o método.

2. Imports y alias

El notebook comienza cargando librerías. Python es un lenguaje de propósito general y amplía sus capacidades mediante paquetes.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Idea para decir en clase: pandas trabaja con datos tabulares; NumPy con cálculo numérico; Matplotlib con gráficos. `pd`, `np` y `plt` son alias para escribir menos.

3. Variables, strings y rutas

Una variable es un nombre que referencia un valor. La ruta del archivo es texto, por lo que es un `str`.

```
ruta = r"C:\...\Afiliados_SNP_Diciembre2025.csv"
type(ruta)
```

Idea para decir en clase: La `r` crea un raw string, útil en rutas de Windows porque evita interpretar las barras invertidas como secuencias especiales.

4. Función, método y objeto

Al leer el CSV, pandas crea un objeto `DataFrame`.

```
df = pd.read_csv(ruta)
type(df)

df.head(10)
```

Idea para decir en clase: `pd.read\_csv(...)` es una función del módulo pandas. `df.head(...)` es un método que pertenece al objeto `df`.

5. Filas, columnas, índices y formato de salida

`shape` devuelve una tupla con dos posiciones: primero filas y luego columnas.

```
df.shape
# (filas, columnas)

df.shape[0] # primera posición -> filas
df.shape[1] # segunda posición -> columnas
```

Idea para decir en clase: Python empieza a contar desde cero. Por eso la primera posición es [0] y la segunda es [1].

El print y las f-strings

```
print(f'Filas: {df.shape[0];,}')
print(f'Columnas: {df.shape[1]}')

print(f'Memoria utilizada: {memoria_mb:.2f} MB')
```

f'...' f-string: permite insertar expresiones dentro de texto.

{...} las llaves indican qué expresión debe evaluarse.

; formato con separador de miles.

:.2f número decimal con dos posiciones.

La coma no significa siempre lo mismo

```
print("Hola", 25)          # separa argumentos
["sexoact", "edadact", "dpto"] # separa elementos
df.iloc[0:5, 0:3]          # separa filas y columnas
```

Idea para decir en clase: La sintaxis se interpreta por contexto. No intenten asignarle a cada símbolo un único significado universal.

6. Listas, corchetes y selección de datos

Una lista es una colección ordenada. Se representa con corchetes y también usa índices que empiezan en cero.

```
columnas = ["sexoact", "edadact", "dpto"]  
columnas[0]  
# "sexoact"
```

Una columna: Series

```
df["sexoact"]  
type(df["sexoact"])
```

Idea para decir en clase: Los corchetes aquí no crean una lista: le piden al DataFrame una columna por su nombre.

Varias columnas: DataFrame

```
df[["sexoact", "edadact", "dpto"]]
```

El corchete exterior hace la selección en `df`; el interior contiene una lista de nombres de columnas.

Los dos puntos: slicing

```
df.iloc[0:5]  
# posiciones 0,1,2,3,4  
  
df.iloc[0:5, 0:3]  
# primeras 5 filas y primeras 3 columnas
```

Idea para decir en clase: En slicing, el límite final no se incluye. `0:5` llega hasta antes de 5.

7. Métodos encadenados y argumentos

Pandas permite encadenar operaciones: el resultado de una operación se convierte en la entrada de la siguiente.

```
df.isnull().sum()
```

Lectura: DataFrame -> detectar nulos -> sumar cuántos hay.

```
df["sexoact"].value_counts(normalize=True) * 100
```

Lectura: seleccionar columna -> contar categorías -> convertir a proporciones -> pasar a porcentaje.

Argumentos posicionales y nombrados

```
df.head(10)
# 10 es un argumento posicional

df.info(memory_usage="deep")
# memory_usage es un argumento nombrado

df["sexoact"].value_counts(normalize=True)
# True es un booleano
```

Idea para decir en clase: Los argumentos nombrados hacen el código más legible: indican explícitamente qué opción estamos configurando.

True y False

```
True
False
ascending=False
normalize=True
```

Son valores booleanos y aparecen tanto en configuraciones de funciones como en filtros lógicos.

8. Diccionarios, comparaciones y filtros booleanos

Diccionario: clave -> valor

```
sexo = {  
    1: "Hombre",  
    2: "Mujer"  
}  
  
sexo[1]
```

Idea para decir en clase: Un diccionario es útil para traducir códigos numéricos a etiquetas comprensibles.

Comparación no es asignación

```
lima = df[df["dpto"] == 15].copy()  
  
= # asignación  
== # comparación
```

La expresión `df["dpto"] == 15` genera una serie de valores True/False. Después, esos booleanos se usan como máscara para conservar solo las filas que cumplen la condición.

```
df["dpto"] == 15  
# False  
# True  
# False  
# ...  
  
df[df["dpto"] == 15]
```

Idea para decir en clase: Antes de enseñar el filtro completo, ejecuta solo la condición. Ver la máscara True/False hace que el concepto quede mucho más claro.

9. GroupBy: del código al análisis

```
promedio_genero = (  
    lima.groupby("genero")["monto_aportes"]  
        .mean()  
        .sort_values(ascending=False)  
)
```

Lectura técnica:

- Agrupar las filas por `genero`.
- Seleccionar la variable `monto\_aportes`.
- Calcular la media dentro de cada grupo.
- Ordenar el resultado.

Idea para decir en clase: Aquí dejamos de hablar solo de sintaxis. Ya estamos expresando una pregunta analítica con código: "¿cuál es el monto promedio por grupo?"

10. Tipos de Python vs tipos de pandas

```
edad = 25          # int  
aporte = 153.40    # float  
ciudad = "Puno"    # str  
activo = True      # bool  
edades = [20, 25, 30] # list  
persona = {"edad": 25} # dict  
forma = (100, 15)   # tuple
```

Pandas añade objetos propios como **DataFrame** y **Series**. Además, cada columna tiene un `dtype`, por ejemplo `int64`, `float64`, `object`, `datetime64` o `bool`.

Idea para decir en clase: Una cosa es el tipo de un objeto de Python y otra el tipo con el que pandas almacena los valores de una columna.

11. Speech sugerido mientras escribes el código

“No vamos a aprender Python como objetivo principal del curso. Vamos a aprender el Python que necesitamos mientras resolvemos un problema real de datos.”

“Primero importamos herramientas. Después creamos variables, cargamos el archivo y obtenemos un DataFrame. Desde ese momento, casi todo el análisis consiste en hacer preguntas sobre ese objeto.”

“El punto nos permite acceder a propiedades o ejecutar métodos. Los paréntesis indican una ejecución y dentro colocamos argumentos.”

“Los corchetes pueden contener una lista o servir para seleccionar información. En pandas, `df["columna"]` significa: dame esa columna.”

“Los índices empiezan en cero. Por eso `shape[0]` representa filas y `shape[1]` columnas. Los dos puntos representan rangos cuando hacemos slicing.”

“Cuando escribimos una condición como `df["dpto"] == 15`, no estamos filtrando todavía: estamos creando una máscara de verdaderos y falsos. El filtro ocurre cuando usamos esa máscara sobre el DataFrame.”

“Finalmente agrupamos, calculamos promedios y visualizamos. Ya estamos usando programación, pero siempre con un propósito analítico.”

Cierre para los estudiantes

Mensaje final: No memoricen comandos aislados. Aprendan a leer el flujo: objeto -> selección -> transformación -> resultado.

12. Hoja rápida de lectura de código

Código	Cómo leerlo
<code>df["col"]</code>	Seleccionar una columna.
<code>df[["a", "b"]]</code>	Seleccionar varias columnas.
<code>df.shape[0]</code>	Número de filas.
<code>df.shape[1]</code>	Número de columnas.
<code>df.head(10)</code>	Mostrar primeras 10 filas.
<code>df.info()</code>	Inspeccionar estructura y tipos.
<code>df.isnull().sum()</code>	Contar nulos por columna.
<code>df["col"].value_counts()</code>	Contar frecuencias.
<code>df[df["col"] == valor]</code>	Filtrar por condición.
<code>df.iloc[0:5, 0:3]</code>	Seleccionar por posiciones.
<code>df.groupby("grupo")["x"].mean()</code>	Promedio de x por grupo.
<code>f"{valor:.2f}"</code>	Insertar y formatear un valor en texto.